B：ROI与面积统计硬件化

设计与操作说明 / ABI3

2533×1900整帧 | 最多17个矩形ROI | PYNQ-Z2

本版设计内容	验收与使用依据
PL反相、严格阈值、ROI并集与逐ROI前景计数。	本文解释设计与操作；构建及实板成绩以实际报告为准。
冻结16ROI与最新17ROI两套参考，配套RoiDriver及Notebook。	先核对文件，再执行明确版本的仿真、构建与板上测试。

CPU提供对齐后的原灰度图；PL输出mask与统计；CPU读取结果并执行后续缺陷规则。
面积统计不能单独代替全部缺陷分类。

2026 年 10 月 31 日
与完整 pcb_b_roi_v03 文件夹配套使用。

目录

1. 这一轮具体增加了什么 3

2. ROI和面积分别是什么意思 3

3. 重叠区域为什么不能直接相加 3

4. 两套金标准怎样使用 3

5. 阈值规则保持一致 4

6. 32位流怎样处理2533列 4

7. 什么时候面积才算有效 4

8. 配置为什么要有回执 5

9. 主要控制和状态 5

10. 实际地址由HWH决定 5

11. 正常程序的调用顺序 6

12. 八个验收案例分别检查什么 6

13. 怎样在Jupyter打开 7

14. 常见问题与下一步边界 7

1. 这一轮具体增加了什么

上一版已经证明电脑准备的数据可以通过真实DMA送进FPGA，经过阈值模块，再完整回到内存。本轮继续利用这条通路，让FPGA在处理一幅图时同时完成最多17个矩形ROI的统计。

数据路径是：CPU提供已经对齐的原始灰度图；DMA送到PL；PL按配置反相、执行严格阈值；PL保留ROI并集中的前景像素，并分别统计每个ROI的前景数；DMA取回图像；CPU读取统计并进行缺陷判定。

反相和ROI输出已经纳入新RTL。发送图像时保持原灰度值，配置中用invert选择反相，由PL执行255-gray。软件参考负责产生对照答案和检查实际结果。

2. ROI和面积分别是什么意思

ROI就是从整幅图中圈出的感兴趣区域。例如一块芯片、一只电容周围的矩形。配置写成(x0,y0,x1,y1)，左上边界包含、右下边界不包含，数组裁剪是image[y0:y1,x0:x1]。

例如矩形(10,20,13,22)，覆盖x=10、11、12和y=20、21，一共6个像素。width对应列数，height对应行数。NumPy形状写成(height,width)，所以本项目原图是(1900,2533)。

这里的前景面积以像素为单位：先执行阈值，输出255的像素称为前景，然后数一个ROI里有多少个前景像素。它不是器件的毫米面积，也不能单独代表器件是否缺失。A的算法还包含模板绝对差、直方图相关等特征；这些分类规则继续由CPU使用。

3. 重叠区域为什么不能直接相加

两个ROI可以重叠。假设两个2×2矩形都覆盖白色像素，并共享一个像素，每个ROI面积都是4，总输出并集面积是7。把两个ROI的4加起来得到8，会重复计算共享像素。

因此硬件保留两类结果：每个ROI独立面积允许同一个像素分别计入多个ROI；输出mask和总前景数按并集计算，同一个像素只出现一次。程序会分别核对这些结果。

4. 两套金标准怎样使用

包中保存两套独立profile：

profile	A配置版本	active ROI数	参考输入的输出白像素
frozen16	v0.1.0	16	314801
latest17	v0.2.0	17	326860

最新配置加入了U4，白色像素增加12059。旧16个ROI的mask与A最初提供的答案逐字节相同。这使得新硬件既能接入最新配置，也能回到原金标准验证一致性。

两套profile的索引顺序不同：最新配置中U4位于index12，后面的索引顺延。对照统计时按当前profile里的ROI名称和索引配对，不能把两版同一个数字索引直接当同一器件。

输入灰度、原答案、生成答案、面积表、ROI配置都带摘要。修改坐标后，需要另生成一个有明确版本的profile和对应答案。软件audit会核对固定文件，防止误改后仍使用原版本的名称。

5. 阈值规则保持一致

PL规则是严格大于：pixel>T输出255，否则0。配置invert=true后，输入g先变为255-g。本轮像素阈值T=120，因此等价于原灰度g<135。

原灰度	反相值	输出是否前景
134	121	是，255
135	120	否，0
136	119	否，0

A配置里的0.896、20.219、27743.5等是比例、模板差或面积的分类门限。硬件控制字中的8位阈值是像素门限，两者用途不同。

6. 32位流怎样处理2533列

一拍AXIS携带4个uint8像素，第一个像素在TDATA低8位。每拍各字节是否有效由TKEEP指定。有效字节按低lane到高lane顺序紧密排列，只有有效字节推进图像坐标。

2533列不能被4整除，但整帧4,812,700字节可以被4整除。一个32位beat可以跨过上一行末尾和下一行开头，保持整图连续排列。整幅图最后一拍才有TLAST。

核心内部把坐标处理、ROI比较和统计累加分段寄存。每拍仍携带最多4个字节，理想无反压时最小输入接收间隔为3个时钟。它使用VALID/READY交接：暂时不能接受输入时拉低READY，上游会保持当前拍，已经发出的输出则保持到下游接收。

7. 什么时候面积才算有效

每包首个有效输入拍清工作计数，然后随着接受的像素累加。输入LAST到达时，最后一拍可能仍在内部计算或等待下游，所以这时不能提前把统计当作已完成结果。

只有输出LAST真正与READY握手时，才把本包工作计数提交为可读快照，并把完成包数加1。软件等DMA结束、接收长度正确、完成包数增加1和busy清零后，再读取面积。

这样即使最后一拍被阻塞、同周期交付上一包并接收下一包，统计仍对应正确的图像。读取计数没有清零副作用；下一包完成后才更新快照。

8. 配置为什么要有回执

新版本沿用一组GPIO提供控制和状态，又增加一组GPIO传递配置数据和统计。写配置时，CPU先写数据，再写地址并翻转一个请求位，随后等待硬件回执位变成同样的值。

可以把它理解为“把坐标放好，敲一下门，等对方确认收到了，再放下一条坐标”。这样能明确知道设置是否生效。忙着处理图像时，硬件拒绝改变几何配置并设置错误位，避免半幅图使用旧坐标、后半幅使用新坐标。

配置过程由RoiDriver封装，普通使用者调用configure即可。不要同时由其他线程、Notebook或另一个DMA驱动改同一组寄存器。

9. 主要控制和状态

控制字段	含义
bits7:0	像素阈值
bit8	旁路，直接输出原有效字节
bit9	流软复位，需置1后再清0
bit10	反相
bit11	输出按ROI并集保留
bit12	配置写入toggle
bits23:16	配置/统计选择地址

状态包含ABI=3、busy、输入包状态、配置回执、配置错误、最近帧尺寸错误以及16位完成包数。控制位和计数地址的完整表在config/INTERFACE_V03.md。

旁路跳过像素变换和前景判定，前景统计固定0，有效输入像素数仍统计。原图本身可能含255，这不改变旁路统计的定义。

10. 实际地址由HWH决定

新工程多了一组GPIO，自动分配地址发生变化。本次创建阶段DMA是0x40400000，cfgGPIO是0x41200000，ctrlGPIO是0x41210000，每段64KiB。最终以配套HWH导出的实际映射为准。

驱动按实例名axi_dma_0、axi_gpio_ctrl、axi_gpio_cfg从HWH解析地址，加载前验证类型、位宽、方向和范围互不重叠。使用本包配套的RoiDriver与同次bit/hwh。ABI0.2的旧驱动与新硬件不是同一接口版本。

11. 正常程序的调用顺序

先在板上PYNQ环境加载本包软件，然后按顺序调用。下面展示接口含义，完整路径和保护检查已经放在Notebook中：

```
driver = RoiDriver.from_bitfile(BITFILE, timeout=10.0)
driver.recover()
driver.configure(2533, 1900, rois, invert=True, roi_enable=True)
actual_mask = driver.transfer(gray, threshold=120, bypass=False)
statistics = driver.read_results()
```

recover确认DMA停止并清流状态；configure写入图像尺寸和ROI；transfer发送原灰度并取回PL结果；read_results读取同一已完成包的统计。传输后由原有安全DMA实现复位并确认双通道Halted，再释放缓冲区。

二维图像必须与配置的(height,width)完全一致。宽高颠倒后即使总字节数相同，也会被拒绝。若传一维字节数组，必须按行连续展开，且长度精确等于width×height。

参数预验证失败时，旧的有效配置保持可用；实际配置中途失败后，驱动会禁止按半套配置传输。确认原因后按受控流程恢复和重新配置。

12. 八个验收案例分别检查什么

顺序	案例	重点
1	32B旁路	最小收发、像素总数
2	64B旁路	连续拍与包数
3	8×4边界与重叠ROI	134/135边界、半开坐标、重叠与禁用
4	640×480旁路	已有图像规模
5	2533×1900旁路	新整图字节顺序与跨行
6	整图反相阈值，输出ROI关闭	全帧阈值及17路面积
7	latest17整图ROI	新并集mask与17路面积
8	frozen16整图ROI	直接对照A原始mask

每例核对图像每个字节、每路ROI面积、总前景、输入像素总数、实际接收长度、包数增量和错误状态。运行中只要一项失败就停下保存证据。

CPU参考耗时、配置耗时、DMA与统计读取耗时分别记录。单次调用包含分配、缓存维护和复位开销，不能把它写成纯PL延迟或持续处理帧率。

13. 怎样在Jupyter打开

完整包应放在：

```
/home/xilinx/jupyter_notebooks/pcb_b_roi_v03
```

进入software，打开ROI_板上逐步验收.ipynb。只数代码格，共6格：配置、软件核对、硬件预检、加载、八案例验证、结果摘要。初始源文件的执行次数为空，RUN_ON_BOARD=False。

先运行第1、2格，确认文件和旧答案一致；准备实际操作后，把第1格改成True，重新运行第1→2→3格，再运行第4→5→6格。第4格会替换板上FPGA配置，第5格自动执行八个案例。

本包会先准备XRT环境再导入PYNQ；通过SSH管理员子进程运行时，也需在导入前提供板卡已有的XILINX_XRT=/usr。完整运行过程和实测结果另存日志，不会用软件参考的通过代替板上通过。

14. 常见问题与下一步边界

现象	核对方向
ABI不匹配或缺cfgGPIO	是否使用本包同次bit/hwh与RoiDriver
文件摘要不匹配	是否完整复制，是否修改过profile或答案
配置回执超时	当前算法是否空闲、是否有其他控制器、配置IP是否对应
单ROI对、总数相加不对	ROI重叠需要分别统计，并集不重复
旧16对、新17差12059	是否混用两版ROI或把U4索引映错
图像正确、计数错误	查看实际stat快照、包数和当前profile索引
640×480照片无法用2533坐标	前端应先对齐/统一尺寸，或另生成匹配坐标配置

若出现PoisonedError，驱动无法确认DMA已经停止，会保留可能仍被访问的缓冲区。保留当前进程和日志，先使硬件在可控条件下停止，再处理恢复。自动执行器也会在这种状态下保持进程，即使诊断文件写入失败也不会主动退出释放缓冲区。

本轮B交付重点是PL算子、仿真、寄存器接口、配套bit/hwh、资源时序和实际数据。相机采集、板面定位、放稳触发、最终缺陷规则和界面属于后续系统集成；应使用同一输入、同一ROI定义与A/C对接。